DevOps Hands-On Project 3: Docker / Docker Compose → Kubernetes with Minikube

SwiftCart Logistics — Kubernetes Implementation Lab

Level: Beginner → Intermediate DevOps
Duration: 1 hours
Platform: Ubuntu 22.04/24.04, Windows + Docker Desktop/WSL2, or macOS with Docker
Cluster: Minikube
Application: Flask + Gunicorn + PostgreSQL
Continuation: Docker Build Lab → Docker Compose Lab → Kubernetes Lab

---

1. Project overview

Project 1 containerized SwiftCart with Docker. Project 2 converted it into a multi-container Docker Compose application. Project 3 migrates that same application architecture to Kubernetes on Minikube.

Students will deploy and demonstrate:

- Pod
- Deployment
- Service
- ConfigMap
- Secret
- PersistentVolumeClaim / Volume
- Ingress
- readiness and liveness probes
- scaling and self-healing
- rolling update and rollback

Case study

SwiftCart Logistics wants the DevOps team to learn Kubernetes before considering a cloud Kubernetes platform. The application already works in Docker Compose, so the exercise is to translate the Compose architecture into Kubernetes resources while keeping the application itself largely unchanged.

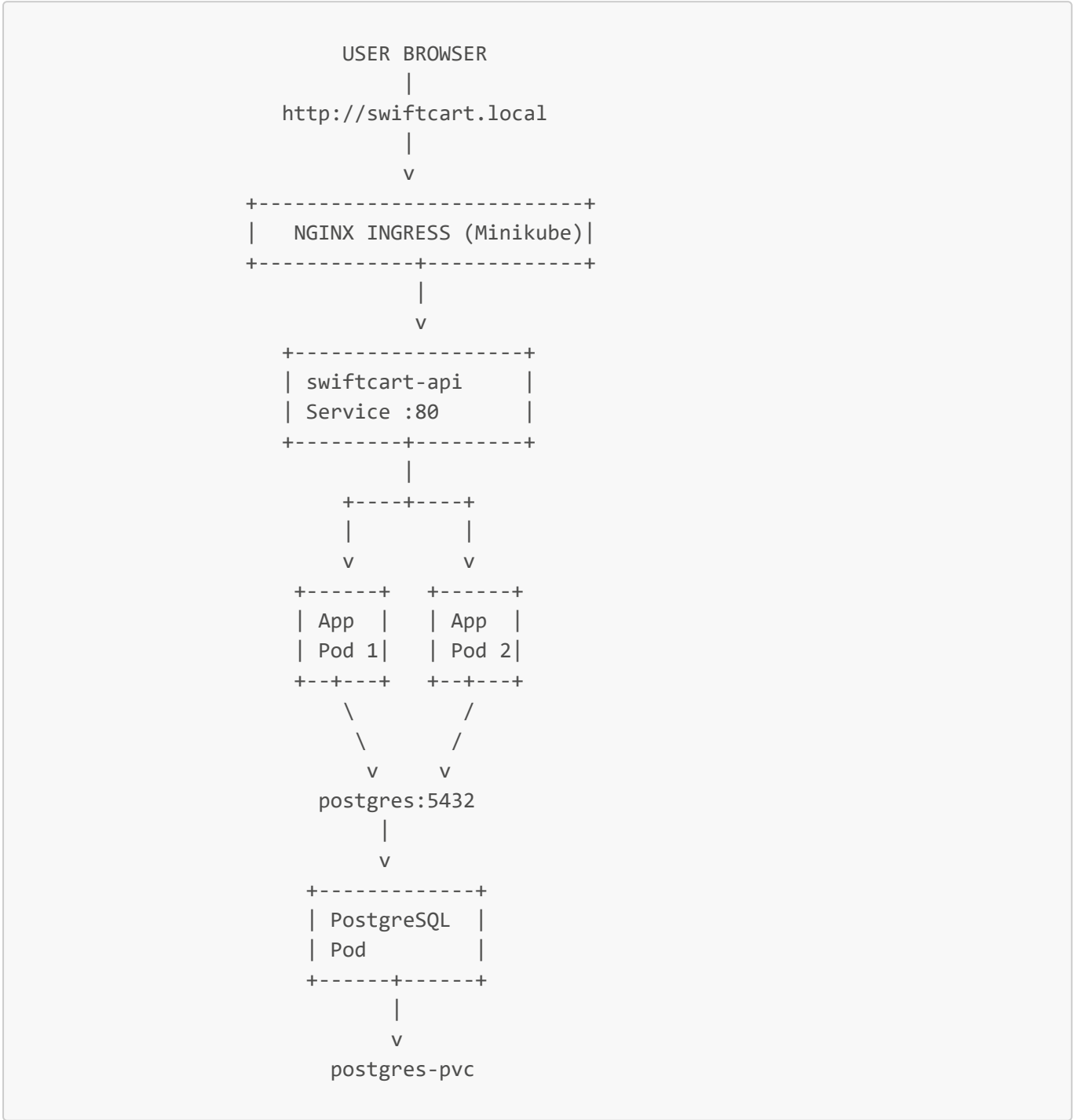
DevOps objectives

1. Start a local Minikube cluster.
2. Build the existing Docker image inside Minikube's Docker environment.
3. Create a Kubernetes namespace.
4. Deploy PostgreSQL with persistent storage.
5. Deploy the SwiftCart application with two replicas.
6. Configure non-sensitive settings with a ConfigMap.
7. Configure credentials with a Secret.
8. Expose workloads through Services.

9. Route browser traffic through an NGINX Ingress controller.
10. Prove service discovery, persistence, self-healing and scaling.
11. Perform a rolling update and rollback.

---

2. Architecture



Configuration flow:

```
ConfigMap ──────► App configuration / DB host / DB name
Secret    ──────► DB username / DB password
PVC       ──────► PostgreSQL persistent data
```

---

3. Docker Compose → Kubernetes mapping

Docker / Compose	Kubernetes
Container	Container inside a Pod
Compose service	Deployment + Service
`environment:`	ConfigMap / Secret / env
Named volume	PVC / Volume
Compose network	Kubernetes networking + Service DNS
`docker compose up`	`kubectl apply`
Compose replicas	Deployment `replicas`
Compose healthcheck	Kubernetes readiness/liveness probes
Published port	Service / Ingress
`depends_on`	readiness probes + application retry/ordering strategy

Kubernetes is not simply a larger Docker Compose file. Kubernetes separates workload management, networking, configuration, secrets and storage into API resources.

---

4. Prerequisites

Install and verify:

```
docker --version
minikube version
kubectl version --client
curl --version
```

Minikube currently documents a baseline of at least 2 CPUs, 2 GB free memory and 20 GB free disk, plus a supported container/VM manager.

---

5. Project files

```
swiftcart-kubernetes-minikube-lab/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .dockerignore
├── k8s/
│   ├── 00-namespace.yaml
│   ├── 01-configmap.yaml
```

```
│   ├── 02-secret.yaml
│   ├── 03-postgres-pvc.yaml
│   ├── 04-postgres-deployment.yaml
│   ├── 05-postgres-service.yaml
│   ├── 06-app-deployment.yaml
│   ├── 07-app-service.yaml
│   ├── 08-ingress.yaml
│   └── optional-pod-demo.yaml
└── README.md
```

---

6. Step-by-step implementation

Step 1 — Start Minikube

```
minikube start --driver=docker
minikube status
kubectl get nodes
kubectl cluster-info
kubectl get pods -A
```

The node should eventually show `Ready`.

Step 2 — Enable the Ingress controller

```
minikube addons enable ingress
minikube addons list | grep ingress
kubectl get pods -n ingress-nginx
```

Wait for the controller:

```
kubectl wait --namespace ingress-nginx --for=condition=Ready pod --all --
timeout=180s
```

An Ingress object by itself does nothing unless an Ingress controller is running.

Step 3 — Build the application image inside Minikube

Use Minikube's Docker environment:

```
eval $(minikube docker-env)
```

Build:

```
docker build -t swiftcart-api:3.0.0 ./app
```

Verify:

```
docker images | grep swiftcart-api
```

Reset the shell later if required:

```
eval $(minikube docker-env -u)
```

This avoids pushing the classroom image to Docker Hub just to run it locally.

Step 4 — Create the namespace

```
kubectl apply -f k8s/00-namespace.yaml
kubectl get namespace swiftcart
```

All project resources use the `swiftcart` namespace.

Step 5 — Create the ConfigMap

```
kubectl apply -f k8s/01-configmap.yaml
kubectl get configmap -n swiftcart
kubectl describe configmap swiftcart-config -n swiftcart
```

It contains normal configuration such as:

```
APP_VERSION=3.0.0
DB_HOST=postgres
DB_PORT=5432
POSTGRES_DB=swiftcart
LOG_LEVEL=INFO
```

Do not put passwords into a ConfigMap.

Step 6 — Create the Secret

```
kubectl apply -f k8s/02-secret.yaml
kubectl get secret -n swiftcart
kubectl describe secret swiftcart-db-secret -n swiftcart
```

The training Secret contains the PostgreSQL username and password.

Security note: Kubernetes Secrets are not automatically a complete secrets-management solution. Production clusters should consider encryption at rest, least-privilege RBAC, access restrictions, rotation and external secret-management systems.

Step 7 — Create persistent storage

```
kubectl apply -f k8s/03-postgres-pvc.yaml
kubectl get pvc -n swiftcart
kubectl describe pvc postgres-pvc -n swiftcart
kubectl get pv
kubectl get storageclass
```

Wait for the PVC to become `Bound`.

Concept:

```
PostgreSQL Pod → PVC → Storage
```

A PVC is a request for persistent storage. The actual backing storage is supplied by the cluster's storage mechanism.

Step 8 — Deploy PostgreSQL

```
kubectl apply -f k8s/04-postgres-deployment.yaml
kubectl get deployment -n swiftcart
kubectl get pods -n swiftcart
kubectl logs deployment/postgres -n swiftcart
```

Watch the Pod:

```
kubectl get pods -n swiftcart -w
```

The database Deployment uses the ConfigMap for the database name and the Secret for credentials, and mounts the PVC.

> **Instructor validation:** PostgreSQL image data-directory conventions can change by major image release. Before class, verify the selected `postgres:18-alpine` image's documented persistent-data path. If its official documentation specifies a different `mountPath` or `PGDATA`, update `04-postgres-deployment.yaml` accordingly. The rest of the lab does not depend on that path.

Step 9 — Create the PostgreSQL Service

```
kubectl apply -f k8s/05-postgres-service.yaml
kubectl get service -n swiftcart
kubectl describe service postgres -n swiftcart
```

The application will use:

```
postgres:5432
```

not a Pod IP.

Step 10 — Understand the Pod

```
kubectl get pods -n swiftcart -o wide
kubectl describe pod -n swiftcart -l app=postgres
```

Pods are ephemeral. A Pod can be replaced and receive a different IP. Services provide the stable networking abstraction.

Optional standalone Pod demonstration

The project includes `k8s/optional-pod-demo.yaml`.

```
kubectl apply -f k8s/optional-pod-demo.yaml
kubectl get pods -n swiftcart
kubectl describe pod swiftcart-pod-demo -n swiftcart
kubectl delete pod swiftcart-pod-demo -n swiftcart
```

Explain why production applications are usually managed through workload resources such as Deployments rather than manually managed standalone Pods.

Step 11 — Deploy the application

```
kubectl apply -f k8s/06-app-deployment.yaml
kubectl get deployment -n swiftcart
kubectl get replicasets -n swiftcart
kubectl get pods -n swiftcart
kubectl rollout status deployment/swiftcart-api -n swiftcart
```

The Deployment specifies two replicas:

```
Deployment
    |
    v
ReplicaSet
  /   \
Pod 1  Pod 2
```

Step 12 — Inspect application logs and health

```
kubectl logs deployment/swiftcart-api -n swiftcart
kubectl get pods -n swiftcart
kubectl describe deployment swiftcart-api -n swiftcart
```

The application has Kubernetes HTTP readiness and liveness probes against:

```
/health
```

Readiness controls whether the Pod is considered ready to receive Service traffic. Liveness helps Kubernetes detect a running container that is no longer healthy.

Step 13 — Create the application Service

```
kubectl apply -f k8s/07-app-service.yaml
kubectl get service swiftcart-api -n swiftcart
kubectl describe service swiftcart-api -n swiftcart
```

The Service selects Pods with:

```
app=swiftcart-api
```

Verify endpoints:

```
kubectl get endpointslices -n swiftcart
```

If a Service has no endpoints, check whether the Service selector exactly matches the Pod labels.

Step 14 — Test service discovery inside the cluster

```
kubectl run curl-test -n swiftcart \
  --image=curlimages/curl:8.10.1 \
  --rm -it --restart=Never -- \
  curl http://swiftcart-api/health
```

Database connectivity through the application:

```
kubectl run curl-test -n swiftcart \
  --image=curlimages/curl:8.10.1 \
  --rm -it --restart=Never -- \
  curl http://swiftcart-api/api/db-check
```

This demonstrates Kubernetes Service DNS and application-to-database connectivity.

Step 15 — Verify ConfigMap and Secret injection

```
kubectl describe pod -n swiftcart -l app=swiftcart-api
```

The application receives normal settings from `swiftcart-config` and database credentials from `swiftcart-db-secret`.

The Docker image itself does not contain the database password.

Step 16 — Verify persistence

Generate visits by opening the application or calling:

```
curl http://swiftcart.local/api/status
```

Record the `visits` value.

Delete the PostgreSQL Pod:

```
kubectl delete pod -n swiftcart -l app=postgres
```

Watch replacement:

```
kubectl get pods -n swiftcart -w
```

After PostgreSQL becomes ready, test again:

```
curl http://swiftcart.local/api/status
```

The data should remain because PostgreSQL is using persistent storage rather than relying only on the Pod's writable container filesystem.

Step 17 — Create the Ingress

```
kubectl apply -f k8s/08-ingress.yaml
kubectl get ingress -n swiftcart
kubectl describe ingress swiftcart-ingress -n swiftcart
```

Traffic flow:

```
Browser
   ↓
swiftcart.local
   ↓
NGINX Ingress Controller
   ↓
swiftcart-api Service
   ↓
Application Pods
```

Step 18 — Configure local hostname

Get the Minikube IP:

```
minikube ip
```

Add an entry to your hosts file using that IP:

```
<MINIKUBE-IP> swiftcart.local
```

Linux/macOS:

```
sudo nano /etc/hosts
```

Windows:

```
C:\Windows\System32\drivers\etc\hosts
```

Test:

```
curl http://swiftcart.local/health
```

Then open:

```
http://swiftcart.local
```

If the Minikube IP is not reachable

Driver/OS behavior can vary. Use port forwarding to verify the application independently:

```
kubectl port-forward service/swiftcart-api 8080:80 -n swiftcart
```

Then:

```
curl http://localhost:8080/health
```

This is a troubleshooting path; still complete the Ingress configuration exercise.

Step 19 — Test the live application

```
curl http://swiftcart.local/
curl http://swiftcart.local/health
curl http://swiftcart.local/api/db-check
curl http://swiftcart.local/api/status
```

The response should identify the application as running on Kubernetes and show the database-backed visit count.

Step 20 — Demonstrate self-healing

List application Pods:

```
kubectl get pods -n swiftcart -l app=swiftcart-api
```

Delete one:

```
kubectl delete pod <APP-POD-NAME> -n swiftcart
```

Watch:

```
kubectl get pods -n swiftcart -w
```

Explain the reconciliation process:

```
Desired state = 2 replicas
        ↓
One Pod disappears
        ↓
Actual state = 1
        ↓
Deployment/ReplicaSet detects difference
        ↓
Replacement Pod created
        ↓
Actual state returns to 2
```

Step 21 — Demonstrate scaling

```
kubectl scale deployment swiftcart-api --replicas=3 -n swiftcart
kubectl get pods -n swiftcart
```

Scale back:

```
kubectl scale deployment swiftcart-api --replicas=2 -n swiftcart
```

Step 22 — Demonstrate rolling update and rollback

Build a new image tag after making a small application change:

```
eval $(minikube docker-env)
docker build -t swiftcart-api:3.0.1 ./app
```

Update the Deployment:

```
kubectl set image deployment/swiftcart-api \
  swiftcart-api=swiftcart-api:3.0.1 \
  -n swiftcart
```

Watch rollout:

```
kubectl rollout status deployment/swiftcart-api -n swiftcart
```

History:

```
kubectl rollout history deployment/swiftcart-api -n swiftcart
```

Rollback:

```
kubectl rollout undo deployment/swiftcart-api -n swiftcart
```

---

7. Troubleshooting

ImagePullBackOff

```
kubectl describe pod <POD-NAME> -n swiftcart
```

If the image was built on the host rather than inside Minikube:

```
eval $(minikube docker-env)
docker build -t swiftcart-api:3.0.0 ./app
kubectl rollout restart deployment/swiftcart-api -n swiftcart
```

Service has no endpoints

```
kubectl get pods -n swiftcart --show-labels
kubectl describe svc swiftcart-api -n swiftcart
kubectl get endpointslices -n swiftcart
```

Confirm both use `app=swiftcart-api`.

App cannot reach PostgreSQL

```
kubectl get pods -n swiftcart
kubectl get svc -n swiftcart
kubectl logs deployment/postgres -n swiftcart
kubectl describe pod -n swiftcart -l app=swiftcart-api
```

The app should use:

```
DB_HOST=postgres
DB_PORT=5432
```

PVC Pending

```
kubectl get pvc -n swiftcart
kubectl describe pvc postgres-pvc -n swiftcart
kubectl get storageclass
```

Ingress not working

```
minikube addons list | grep ingress
kubectl get pods -n ingress-nginx
```

```
kubectl get ingress -n swiftcart
kubectl describe ingress swiftcart-ingress -n swiftcart
minikube ip
```

Check the hosts file and ensure `swiftcart.local` resolves to the appropriate Minikube address.

General evidence

```
kubectl get all -n swiftcart
kubectl get events -n swiftcart --sort-by=.lastTimestamp
```

---

8. Assessment / student practical tasks

Task 1 — Architecture

Draw and explain:

```
Browser → Ingress → Service → App Pods → PostgreSQL Service → PostgreSQL Pod → PVC
```

Task 2 — Scale

Scale to five replicas and capture the result:

```
kubectl scale deployment swiftcart-api --replicas=5 -n swiftcart
kubectl get pods -n swiftcart
```

Task 3 — Self-healing

Delete two application Pods and explain why replacements appear.

Task 4 — ConfigMap

Change `LOG_LEVEL` from `INFO` to `DEBUG`. Explain why normal configuration belongs in a ConfigMap rather than the Dockerfile.

Task 5 — Persistence

Delete PostgreSQL's Pod and prove the visit count survives the replacement.

Task 6 — Service discovery

Explain why the app uses `postgres:5432` instead of a Pod IP.

Task 7 — Ingress

Explain why the browser reaches the Ingress and not an individual Pod.

Task 8 — Troubleshooting

Temporarily change the application Service selector so it no longer matches the Pods. Observe the empty endpoints, then fix the selector.

---

9. Security discussion

This is a classroom lab. The sample database password is intentionally simple.

For production, discuss:

- encryption at rest for Secrets
- RBAC and least privilege
- external secret managers
- credential rotation
- TLS
- NetworkPolicies
- image vulnerability scanning
- non-root containers
- resource requests and limits
- logging and audit trails
- backup and disaster recovery
- pinned image versions/digests

---

10. Ingress note

Ingress is a stable Kubernetes API, but the Kubernetes project currently recommends Gateway API for newer advanced traffic-management designs. Ingress remains useful for this learning project and is supported by Minikube's ingress addon.

---

11. Final validation checklist

Capture evidence for:

```
minikube status
kubectl get nodes
kubectl get namespace swiftcart
kubectl get configmap -n swiftcart
kubectl get secret -n swiftcart
```

```
kubectl get pvc -n swiftcart
kubectl get deployment -n swiftcart
kubectl get pods -n swiftcart
kubectl get svc -n swiftcart
kubectl get ingress -n swiftcart
kubectl get endpointslices -n swiftcart
```

Live application:

```
curl http://swiftcart.local/health
curl http://swiftcart.local/api/db-check
curl http://swiftcart.local/api/status
```

Self-healing:

```
kubectl delete pod <APP-POD-NAME> -n swiftcart
kubectl get pods -n swiftcart -w
```

---

12. Lessons learned

1. Docker packages and runs the application container.
2. Docker Compose coordinates a local multi-container environment.
3. Kubernetes manages containerized workloads declaratively.
4. Pods are the smallest deployable units.
5. Deployments maintain the desired number of application replicas.
6. Services provide stable networking and service discovery.
7. ConfigMaps hold non-sensitive configuration.
8. Secrets hold sensitive configuration, with additional security controls required in production.
9. PVCs provide persistent storage independently of an individual Pod's lifecycle.
10. Ingress routes HTTP/HTTPS traffic into Services.
11. Kubernetes controllers continuously reconcile desired and actual state.
12. Minikube provides a practical local Kubernetes environment for learning and development.

---

13. Command cheat sheet

```
# Minikube
minikube start --driver=docker
minikube status
minikube ip
minikube addons enable ingress
minikube addons list
```

```
minikube stop
minikube delete

# Kubernetes inspection
kubectl get nodes
kubectl get all -n swiftcart
kubectl get pods -n swiftcart
kubectl get svc -n swiftcart
kubectl get deployments -n swiftcart
kubectl get pvc -n swiftcart
kubectl get ingress -n swiftcart

# Apply
kubectl apply -f k8s/<file>.yaml

# Logs
kubectl logs deployment/swiftcart-api -n swiftcart
kubectl logs deployment/postgres -n swiftcart

# Inspect
kubectl describe pod <pod> -n swiftcart
kubectl describe svc swiftcart-api -n swiftcart
kubectl describe ingress swiftcart-ingress -n swiftcart

# Scale
kubectl scale deployment swiftcart-api --replicas=3 -n swiftcart

# Rollouts
kubectl rollout status deployment/swiftcart-api -n swiftcart
kubectl rollout history deployment/swiftcart-api -n swiftcart
kubectl rollout undo deployment/swiftcart-api -n swiftcart

# Port forwarding
kubectl port-forward service/swiftcart-api 8080:80 -n swiftcart

# Minikube Docker environment
eval $(minikube docker-env)
docker build -t swiftcart-api:3.0.0 ./app
eval $(minikube docker-env -u)

# Cleanup
kubectl delete namespace swiftcart
minikube delete
```

---

14. References

Kubernetes

- Pods: https://kubernetes.io/docs/concepts/workloads/pods/
- Pod lifecycle: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/

- Deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Services: https://kubernetes.io/docs/concepts/services-networking/service/
- Services and networking: https://kubernetes.io/docs/concepts/services-networking/
- ConfigMaps: https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/
- Secrets: https://kubernetes.io/docs/concepts/configuration/secret/
- Volumes: https://kubernetes.io/docs/concepts/storage/volumes/
- Persistent Volumes / PVC: https://kubernetes.io/docs/concepts/storage/persistent-volumes/
- Ingress: https://kubernetes.io/docs/concepts/services-networking/ingress/
- Ingress controllers: https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/

Minikube

- Start: https://minikube.sigs.k8s.io/docs/start/
- Deploying apps: https://minikube.sigs.k8s.io/docs/handbook/deploying/
- Addons: https://minikube.sigs.k8s.io/docs/handbook/addons/
- Docker driver: https://minikube.sigs.k8s.io/docs/drivers/docker/
- Ingress: https://minikube.sigs.k8s.io/docs/tutorials/nginx_tcp_udp_ingress/
- Tunnel: https://minikube.sigs.k8s.io/docs/commands/tunnel/

Docker

- Docker Engine: https://docs.docker.com/engine/
- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker Compose: https://docs.docker.com/compose/

Instructor roadmap

```
Project 1 — Docker
       ↓
Project 2 — Docker Compose
       ↓
Project 3 — Kubernetes + Minikube
       ↓
Project 4 — Kubernetes + NGINX / TLS / advanced Ingress
       ↓
Project 5 — Kubernetes CI/CD with GitHub Actions
       ↓
Project 6 — Prometheus + Grafana
       ↓
Project 7 — Helm
       ↓
Project 8 — Managed Kubernetes / Cloud deployment
```

Instructor note: StatefulSet is intentionally not introduced in this project. The database is deployed with a single-replica Deployment so students can focus on the requested Kubernetes fundamentals. StatefulSet should be taught as a subsequent topic for stateful workloads.